

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
ESCOLA SUPERIOR POLITÉCNICA
ENGENHARIA DE SOFTWARE
DESENVOLVIMENTO WEB BACK END

ATIVIDADE PRÁTICA
API REST – BAOZI STORE

RODRIGO SOARES – RU: 3585667

SUA CIDADE – SEU ESTADO

2026

ATIVIDADE PRÁTICA

1. Descrição de uma situação fictícia

A Baozi Store é uma pequena loja que vende pão chinês. Para melhorar a organização do negócio, foi criado um sistema simples para controlar clientes, produtos e pedidos.

Um cliente chamado **Rodrigo3585667** realizou seu cadastro no sistema em 09/11/2026.

O produto vendido pela loja chama-se **Baozi Tradicional** e é vendido por unidade, ao preço de **R$ 6,50**.

Em um determinado momento, o cliente realizou um pedido de **10 unidades** do produto. Mais tarde, o cliente pediu para aumentar o pedido para **12 unidades**, e a alteração foi feita pelo endpoint de atualização da API.

O sistema registra o cliente, o produto comprado e a quantidade solicitada, facilitando o controle da loja.

Os dados desta situação (cliente `Rodrigo3585667`, produto `Baozi Tradicional` e o pedido de 10 unidades) aparecem nos prints do Postman do item 4.

2. Diagrama caso de uso

O diagrama abaixo, em notação UML, mostra o ator principal (**Usuário da API**) e os casos de uso implementados na API para Cliente, Produto e Pedido. Os casos *Registrar Pedido* e *Atualizar Pedido* incluem («include») a validação de que o cliente e o produto informados existem no cadastro.

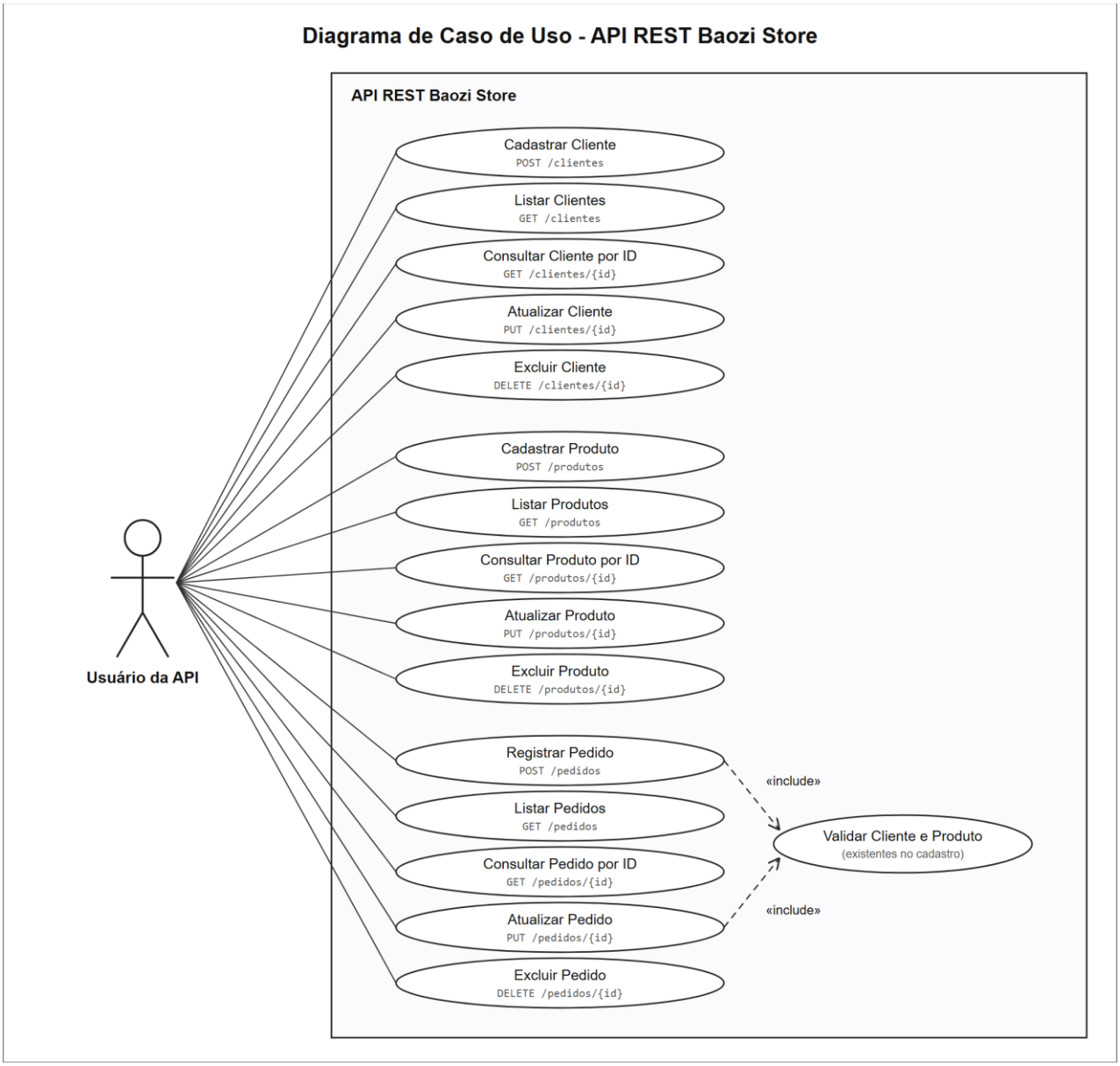


Figura 1 – Diagrama de Caso de Uso da API REST Baozi Store

3. Especificação da API desenvolvida

3.1 Tecnologias

Item	Utilizado
Linguagem	Java 21
Framework	Spring Boot 4.1.2 (Spring Web MVC)
Persistência	Spring Data JPA com Hibernate
Banco de dados	MySQL 8.0 (banco `baozistore`)
Formato dos dados	JSON
Build	Maven
Testes	Postman

3.2 Organização do projeto (padrão MVC do Spring)

Pacote	Conteúdo
`br.com.baozistore.model`	Entidades JPA: `Cliente`, `Produto` e `Pedido`
`br.com.baozistore.repository`	Interfaces `JpaRepository` de cada entidade
`br.com.baozistore.controller`	Controllers REST (`@RestController`) com os endpoints e o `PedidoRequest` (corpo JSON do pedido)

3.3 Entidades e campos

Cliente (tabela cliente)

Campo	Tipo Java	Coluna	Descrição
id	Long	id (PK, auto incremento)	Identificador do cliente
nome	String	nome	Nome do cliente
clienteDesde	LocalDate	cliente_desde	Data de cadastro do cliente

Produto (tabela produto)

Campo	Tipo Java	Coluna	Descrição
id	Long	id (PK, auto incremento)	Identificador do produto
nome	String	nome	Nome do produto
preco	BigDecimal	preco DECIMAL(10,2)	Preço da unidade
estoque	Boolean	estoque	Se há o produto em estoque

Pedido (tabela pedido)

Campo	Tipo Java	Coluna	Descrição
id	Long	id (PK, auto incremento)	Identificador do pedido
cliente	Cliente (`@ManyToOne`)	cliente_id (FK)	Cliente que comprou
produto	Produto (`@ManyToOne`)	produto_id (FK)	Produto comprado
quantidade	Integer	quantidade	Quantidade comprada

Um cliente pode ter vários pedidos e um produto pode estar em vários pedidos; cada pedido tem um único cliente e um único produto, como no DER do enunciado. Para criar ou atualizar um pedido, o JSON enviado usa os campos `clienteId`, `produtoId` e `quantidade`.

3.4 Endpoints implementados

URL base: `http://localhost:8080`. Todos os corpos de requisição e resposta usam JSON.

Método	Caminho	Descrição	Respostas
POST	`/clientes`	Cadastra um cliente	201
GET	`/clientes`	Lista todos os clientes	200
GET	`/clientes/{id}`	Consulta um cliente pelo id	200, 404
PUT	`/clientes/{id}`	Atualiza um cliente (opcional)	200, 404
DELETE	`/clientes/{id}`	Apaga um cliente	204, 404, 409*
POST	`/produtos`	Cadastra um produto	201
GET	`/produtos`	Lista todos os produtos	200
GET	`/produtos/{id}`	Consulta um produto pelo id	200, 404
PUT	`/produtos/{id}`	Atualiza um produto (opcional)	200, 404
DELETE	`/produtos/{id}`	Apaga um produto	204, 404, 409*
POST	`/pedidos`	Registra um pedido	201, 400**
GET	`/pedidos`	Lista todos os pedidos	200
GET	`/pedidos/{id}`	Consulta um pedido pelo id	200, 404
PUT	`/pedidos/{id}`	Atualiza um pedido (opcional)	200, 400**, 404
DELETE	`/pedidos/{id}`	Apaga um pedido	204, 404

* 409 Conflict: o cliente ou produto ainda tem pedidos e não pode ser apagado.
** 400 Bad Request: cliente ou produto inexistente, ou quantidade menor ou igual a zero.

3.5 Exemplos de corpo JSON

```
POST /clientes  { "nome": "Rodrigo3585667", "clienteDesde": "2026-11-09" }
POST /produtos  { "nome": "Baozi Tradicional", "preco": 6.50, "estoque": true }
POST /pedidos   { "clienteId": 1, "produtoId": 1, "quantidade": 10 }
```

3.6 Configuração do banco

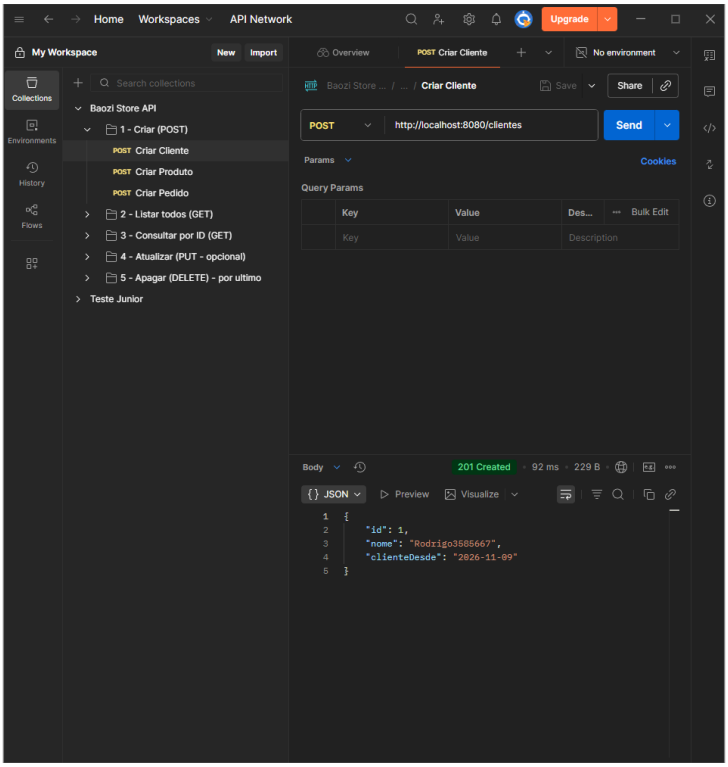
O banco MySQL é configurado no arquivo `src/main/resources/application.properties`. O banco `baozistore` é criado automaticamente na primeira execução e as tabelas são geradas a partir das entidades (`ddl-auto=update`). A senha fica num arquivo local fora do repositório.

```
spring.datasource.url=jdbc:mysql://localhost:3306/baozistore?createDatabaseIfNotExist=true
spring.jpa.hibernate.ddl-auto=update
```

4. Prints do Postman

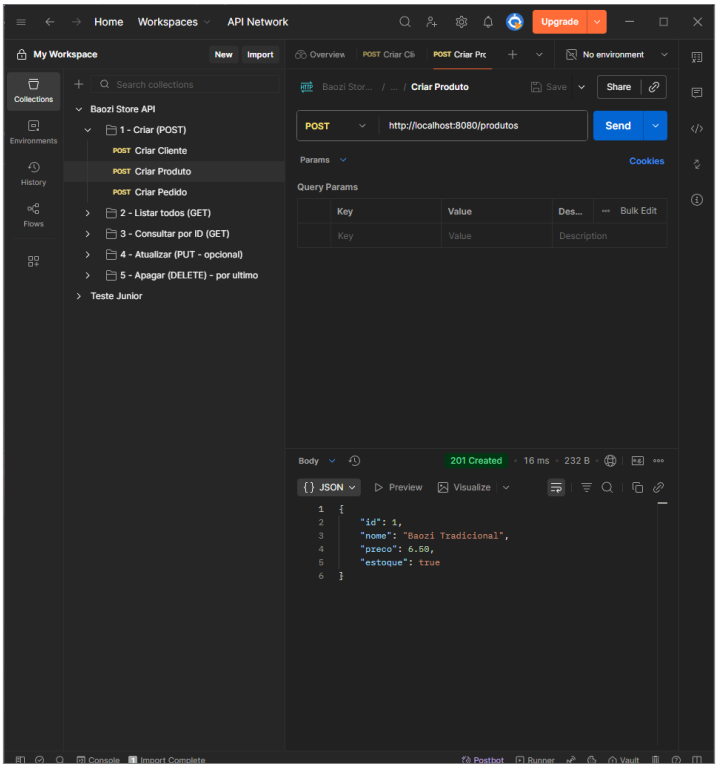
Testes feitos no Postman com a API rodando localmente, usando os dados da situação fictícia do item 1.

Figura 2 – POST /clientes – Criar Cliente



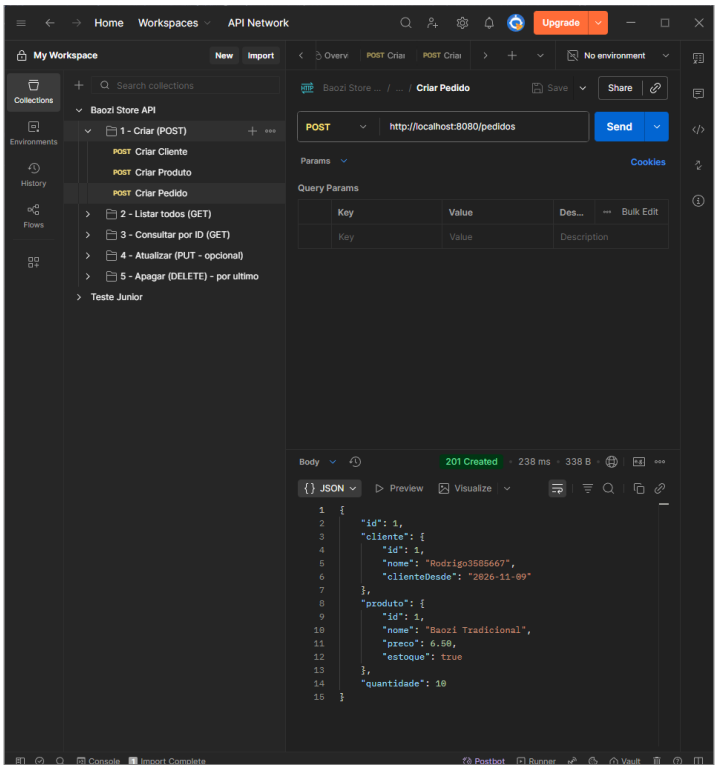
Cadastro do cliente Rodrigo3585667. A API devolve 201 Created com o id gerado pelo banco.

Figura 3 – POST /produtos – Criar Produto



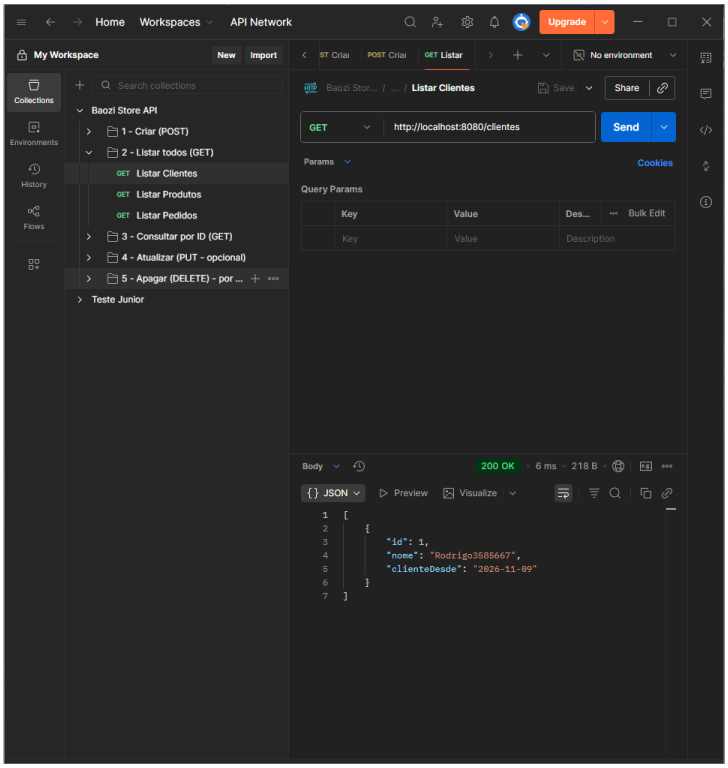
Cadastro do produto Baozi Tradicional, vendido por unidade a R$ 6,50.

Figura 4 – POST /pedidos – Criar Pedido



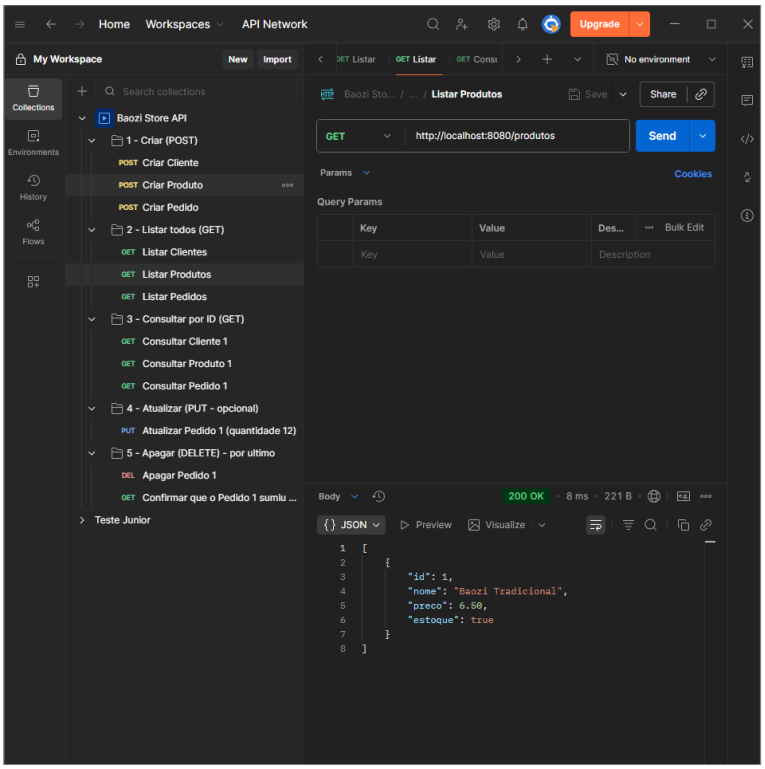
Pedido de 10 unidades do produto 1 pelo cliente 1. A resposta traz o cliente e o produto completos.

Figura 5 – GET /clientes – Listar todos os clientes



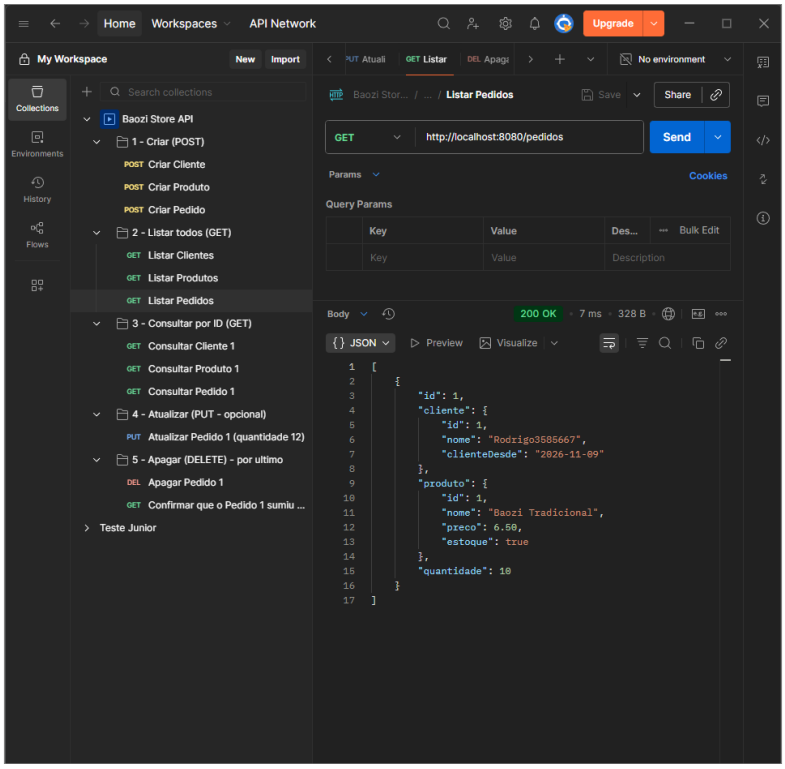
Listagem geral dos clientes cadastrados.

Figura 6 – GET /produtos – Listar todos os produtos



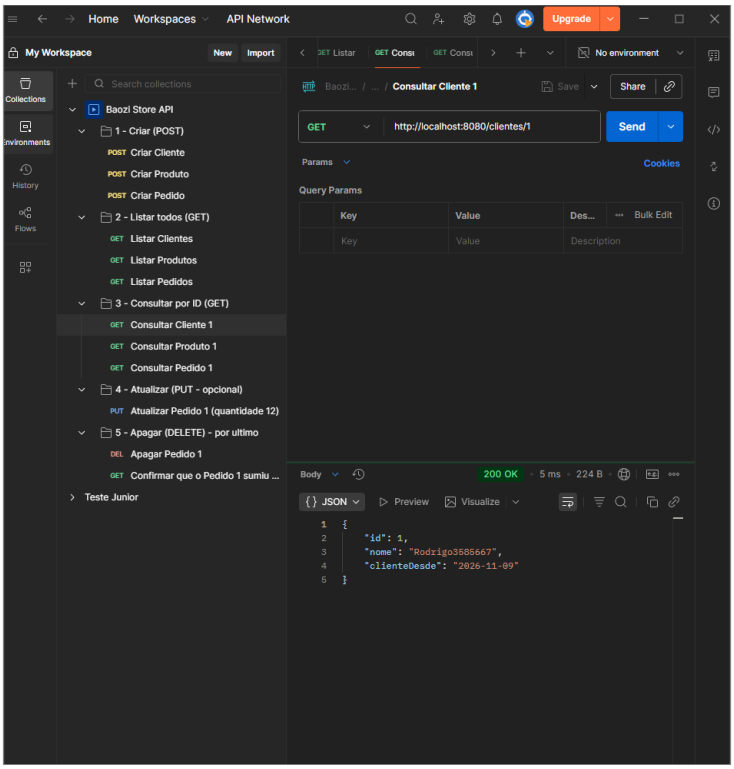
Listagem geral dos produtos cadastrados.

Figura 7 – GET /pedidos – Listar todos os pedidos



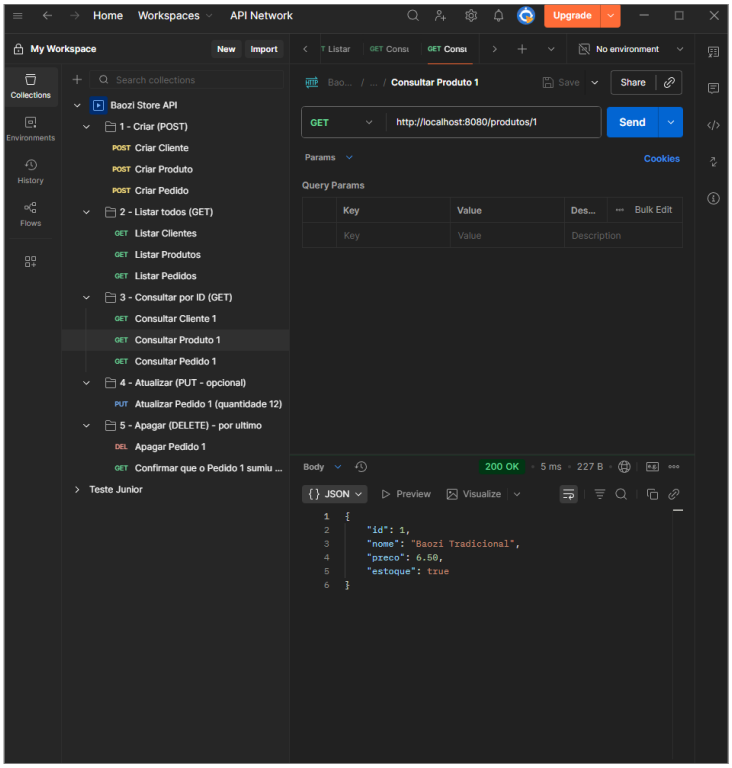
Listagem geral dos pedidos registrados.

Figura 8 – GET /clientes/1 – Consultar cliente por ID



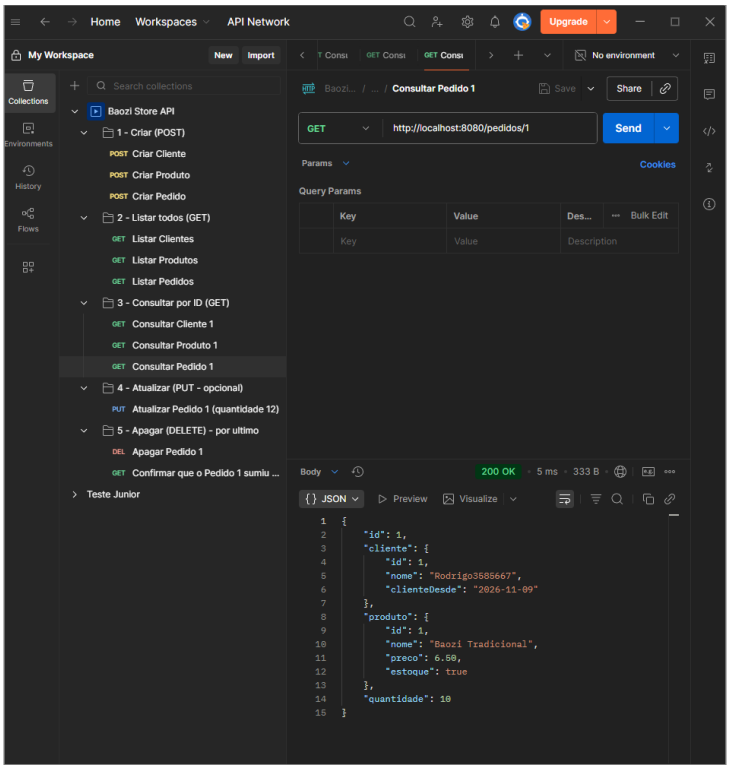
Consulta do cliente de id 1.

Figura 9 – GET /produtos/1 – Consultar produto por ID



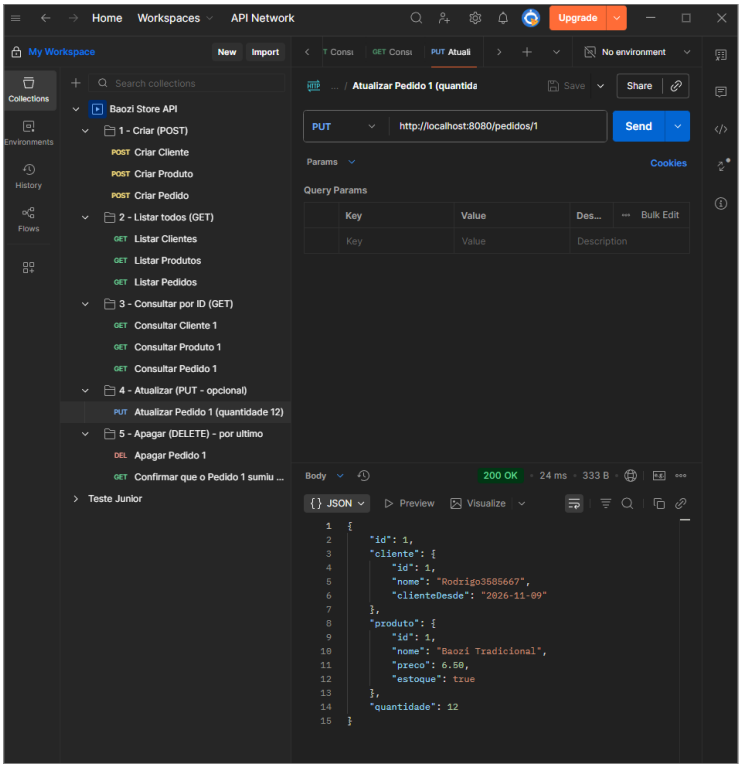
Consulta do produto de id 1.

Figura 10 – GET /pedidos/1 – Consultar pedido por ID



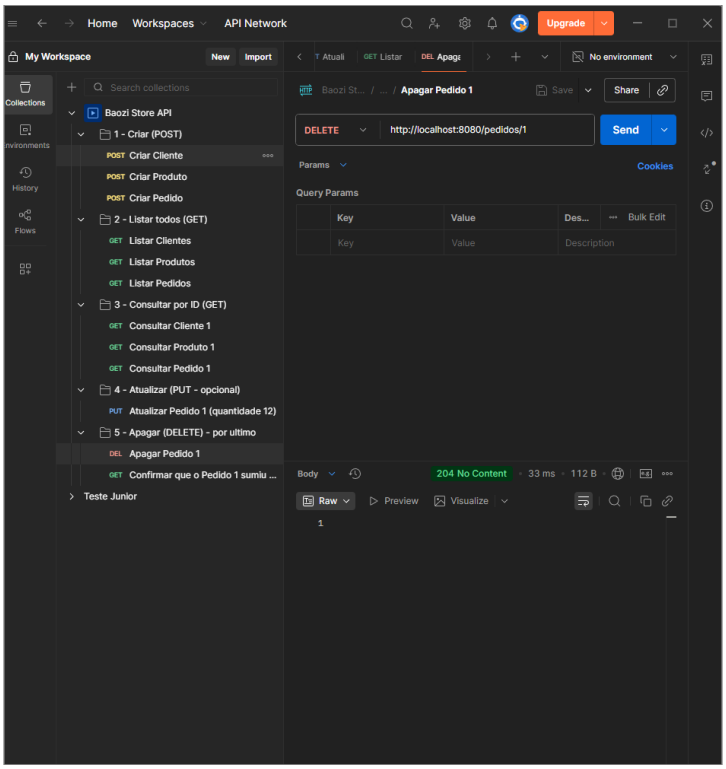
Consulta do pedido de id 1, ainda com 10 unidades.

Figura 11 – PUT /pedidos/1 – Atualizar pedido (opcional)



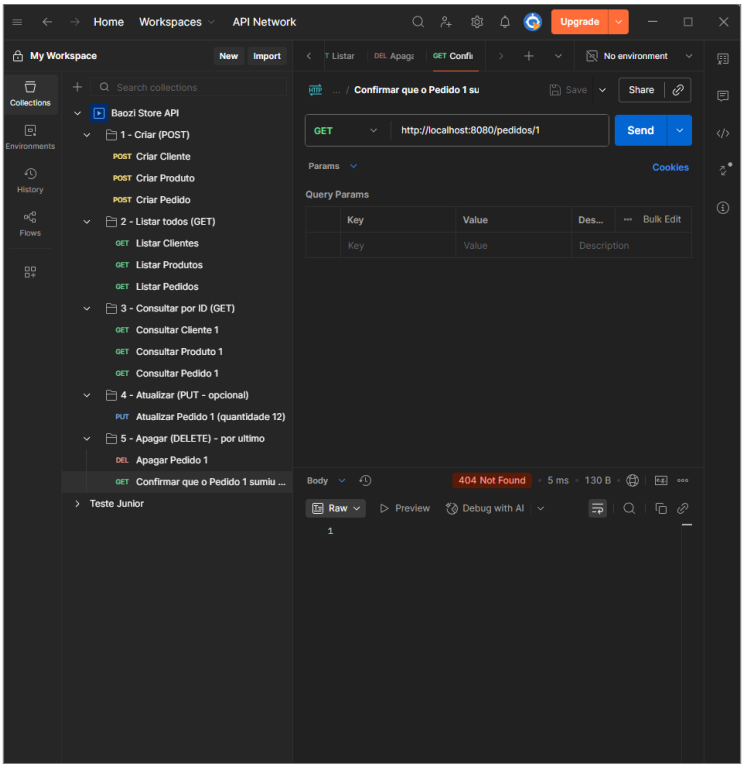
Atualização da quantidade do pedido de 10 para 12 unidades.

Figura 12 – DELETE /pedidos/1 – Apagar pedido



Exclusão do pedido de id 1. A API responde 204 No Content.

Figura 13 – GET /pedidos/1 – Confirmação da exclusão



Depois do DELETE, a consulta do pedido 1 responde 404 Not Found.

5. Link do repositório (GitHub ou similar) contendo o projeto (código fonte)

Código-fonte completo: [link do repositório a preencher]